

PROJECT REPORT

EXPERIMENT – 4

Problem statement:

Design and simulate an Air Pollution Monitoring System using MQ2 Gas Sensor. The system detects harmful gases and monitors air quality levels for industrial and environmental safety applications.

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

Design and Simulation of an Air Pollution Monitoring System Using MQ2 Gas Sensor

Project Overview

The Air Pollution Monitoring System Using MQ2 Gas Sensor is designed to detect harmful gases and smoke in the surrounding environment and provide real-time air-quality monitoring. The system uses an ESP8266 Node MCU as the main controller, which receives the gas-level signal from the MQ2 Gas Sensor and processes it to determine the pollution condition.

The detected gas level and status are displayed on an LCD display. Two LED indicators provide visual alerts, where the green LED indicates normal air conditions and the red LED indicates high pollution levels. A buzzer provides an audible warning when the detected gas level exceeds the predefined safety threshold.

This system provides a low-cost and simple solution for continuous gas and smoke monitoring, making it suitable for basic industrial, laboratory, indoor, and environmental safety applications.

Main Objectives:

- To design a low-cost air pollution monitoring system.
- To detect smoke and combustible gases using an MQ2 sensor.
- To continuously monitor the gas level in the surrounding environment.
- To display the detected gas level on an LCD display.
- To provide visual indication using two LEDs.
- To provide an audible warning using a buzzer.
- To use ESP8266 Node MCU for sensor processing and system control.
- To improve awareness of potentially unsafe air conditions.

Problem Statement

Air pollution caused by harmful gases and smoke can create unsafe conditions in industrial, indoor, and environmental areas. Traditional monitoring methods may not provide continuous monitoring or immediate alerts when pollution levels increase. Therefore, there is a need for a low-cost, real-time air pollution monitoring system that can detect harmful gases and provide an immediate warning.

This project proposes an Air Pollution Monitoring System using an MQ2 Gas Sensor and ESP8266 Node MCU to continuously monitor gas levels. The system displays the detected level on an LCD display and activates LED indicators and a buzzer when the gas level exceeds the predefined threshold, helping users identify potentially unsafe air conditions quickly.

Problems Addressed

The proposed Air Pollution Monitoring System addresses the following problems:

- Lack of continuous monitoring of harmful gases and smoke.
- Delayed detection of increased pollution levels.
- No immediate warning when gas levels become unsafe.
- Difficulty in identifying air-quality changes through manual observation.
- Lack of low-cost monitoring solutions for small-scale applications.
- Insufficient visual and audible alerts during high pollution conditions.
- Need for real-time monitoring in industrial, indoor, and environmental areas.
- Limited awareness of harmful gas presence in the surrounding environment.

Proposed Operation:

The proposed system continuously monitors the surrounding air using the MQ2 Gas Sensor. The sensor detects smoke and combustible gases and sends an analog signal to the ESP8266 Node MCU. The Node MCU processes the sensor reading and compares it with predefined pollution thresholds.

- When the gas level is normal, the green LED remains ON and the buzzer remains OFF.
- When the gas level increases to the warning range, the red LED turns ON and the buzzer is activated.
- When the gas level reaches the danger range, the red LED and buzzer remain active to provide an immediate warning.
- The LCD display continuously shows the gas sensor reading and the corresponding air-quality status.
- The monitoring process repeats continuously to provide real-time pollution detection and alerting.

System Architecture

The system architecture is divided into three main sections: Input Section, Processing & Control Section, and Output Section.

Input Section:

The MQ2 Gas Sensor acts as the input device. It continuously detects smoke and combustible gases present in the surrounding air and generates an analog signal corresponding to the detected gas level.

- MQ2 Gas Sensor – Detects smoke and harmful/combustible gases.
- Sensor output – Analog gas-level signal.

Processing & Control Section:

The ESP8266 Node MCU acts as the main processing and control unit. It receives the analog signal from the MQ2 sensor, processes the sensor reading, and Reads MQ2 sensor value.

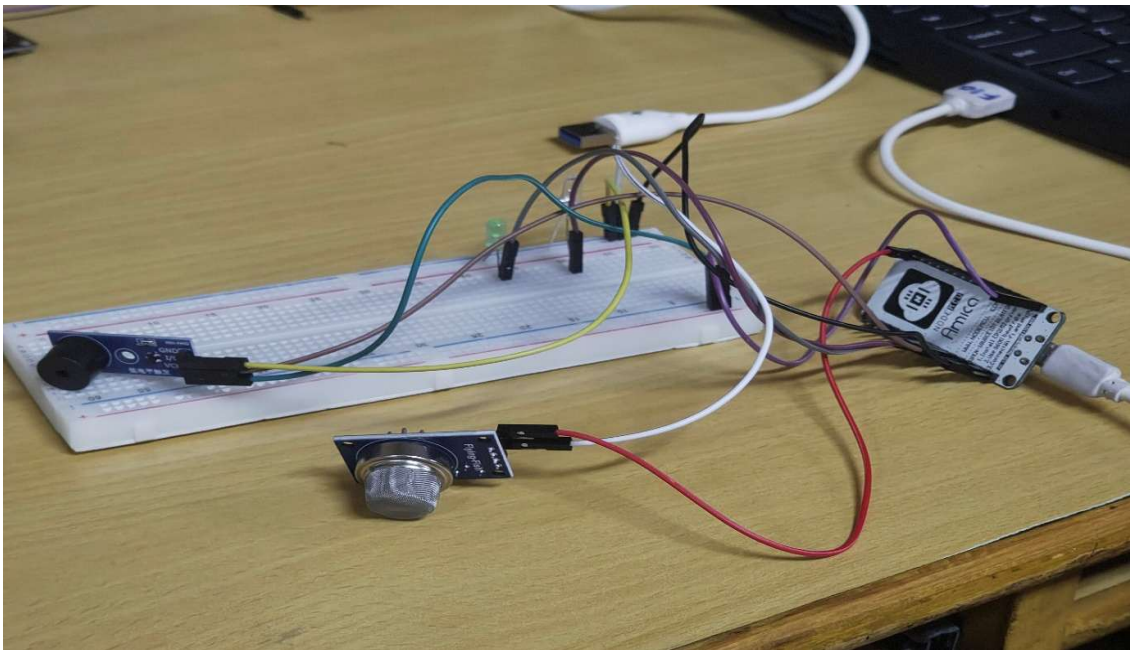
- Processes the sensor data.
- Compares the value with threshold levels.
- Determines Normal, Warning, or Danger condition.
- Controls the display, LEDs, and buzzer.

Output Section:

The output section provides information and alerts to the user based on the processed gas level.

- LCD Display – Displays gas level and pollution status.
- Green LED – Indicates normal condition.
- Red LED – Indicates high pollution/warning condition.
- Buzzer – Provides an audible alert when the gas level exceeds the set threshold.

Image:



Circuit table

S. No.	Component	Component Pin	ESP8266 Node MCU Pin	Function
1	MQ2 Gas Sensor	VCC	VIN / 5V	Sensor power
		GND	GND	Common ground
		AO	A0	Gas-level analog input
2	Green LED	Anode	D5 (GPIO14)	Normal indication
		Cathode	GND	Ground
3	Red LED	Anode	D6 (GPIO12)	Warning indication
		Cathode	GND	Ground
4	Buzzer	Positive	D7 (GPIO13)	Alarm output
		Negative	GND	Ground

For a real hardware build, LED current-limiting resistors are normally required, and the MQ2 analog output must be kept within the Node MCU ADC's permitted voltage range. The exact ADC limit depends on the particular ESP8266 development board. If the MQ2 module's AO can exceed that limit, an appropriate voltage divider/interface is required.

Important Pin Mapping:

MQ2 → A0

OLED SDA → D2

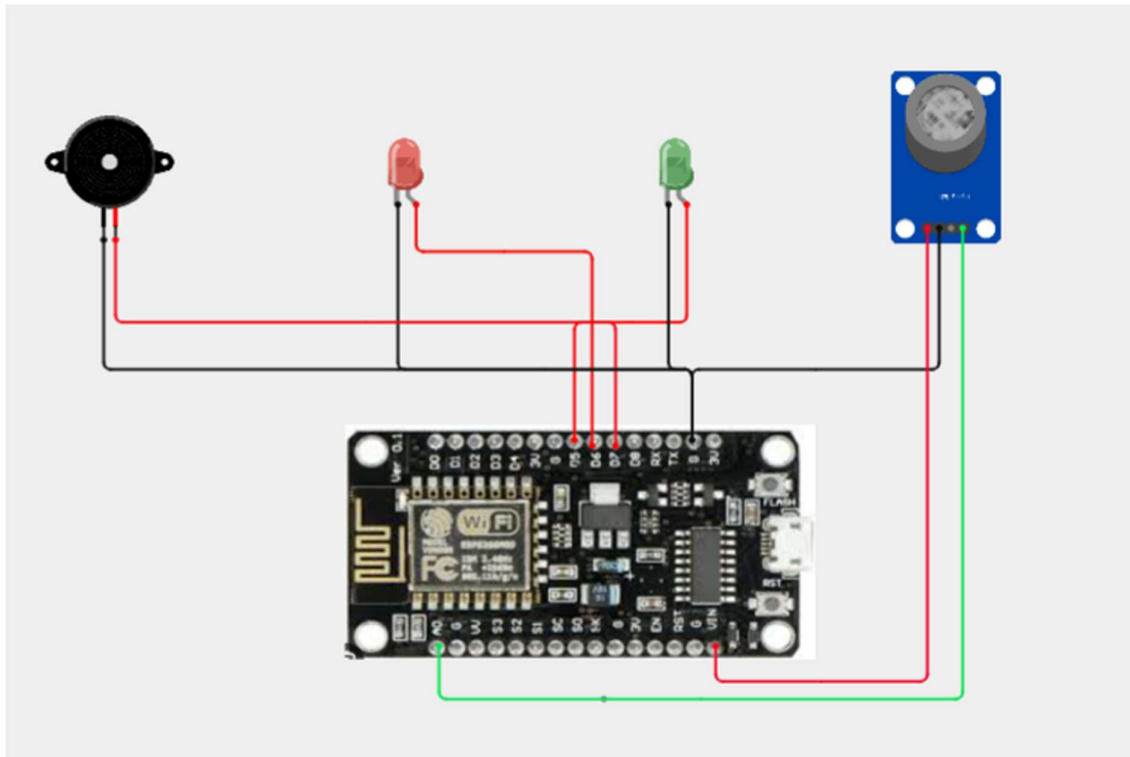
OLED SCL → D1

Green LED → D5

Red LED → D6

Buzzer → D7

Circuit Design



Algorithm

Step 1: Start the system.

Step 2: Power ON the ESP8266 Node MCU and all connected components.

Step 3: Initialize the MQ2 Gas Sensor, LCD display, LEDs, and buzzer.

Step 4: Allow the MQ2 sensor to warm up and stabilize.

Step 5: Read the analog gas value from the MQ2 sensor through the **A0 pin**.

Step 6: Send the sensor reading to the ESP8266 for processing.

Step 7: Compare the sensor reading with the predefined threshold values.

Step 8: If the gas level is normal, turn ON the green LED and turn OFF the red LED and buzzer.

Step 9: If the gas level reaches the warning range, turn ON the red LED and buzzer and display "WARNING".

Step 10: If the gas level reaches the danger range, keep the red LED and buzzer ON and display “DANGER”.

Step 11: Display the current gas sensor reading and air-quality status on the LCD.

Step 12: Continuously repeat the sensor reading and decision process for real-time monitoring.

Step 13: Stop the system when the power supply is switched OFF.

Coding

```
#include <ESP8266WiFi.h>
```

```
#include <ThingSpeak.h>
```

```
// --- Wi-Fi Credentials ---
```

```
const char* ssid = "Rithick";
```

```
const char* password = "12345678";
```

```
// --- ThingSpeak Configuration ---
```

```
unsigned long myChannelNumber = 3458771;
```

```
const char * myWriteAPIKey = "M9YWBVM273R9CG3D";
```

```
// --- Sensor Configuration ---
```

```
const int AIR_QUALITY_PIN = A0; // MQ-135 Analog Pin
```

```
// AQI Thresholds (Raw ADC values 0-1023)
```

```
const int AQI_GOOD_LIMIT = 300; // Clean Air / Normal background
```

```
const int AQI_MODERATE_LIMIT = 600; // Acceptable indoor air / slight  
pollutants
```

```
WiFiClient client;
```

```
// Helper function to convert AQI raw value into textual status
```

```
String getAirQualityStatus(int rawVal) {  
    if (rawVal < AQI_GOOD_LIMIT) {  
        return "Good Air Quality";  
    } else if (rawVal < AQI_MODERATE_LIMIT) {  
        return "Moderate Air Quality";  
    } else {  
        return "POOR / POLLUTED AIR";  
    }  
}
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    Serial.println("=====
```

```
    Serial.println(" IoT AIR QUALITY MONITORING SYSTEM (MQ-135) ");
```

```
    Serial.println("=====
```

```
// Connect to Wi-Fi
```

```
Serial.print("[WIFI] Connecting to ");
```

```
Serial.println(ssid);
```

```
WiFi.begin(ssid, password);
```

```
while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
}  
Serial.println("\n[WIFI] Connected successfully.");  
  
// Initialize ThingSpeak API  
ThingSpeak.begin(client);  
  
Serial.println("[SENSOR] Warming up air quality heater element...");  
delay(20000); // 20-second initial warm-up (3-5 mins recommended for  
production)  
Serial.println("[SENSOR] Sensor online. Starting sampling loop.\n");  
}  
  
void loop() {  
    // 1. Read Raw Air Quality Sensor Data  
    int rawADC = analogRead(AIR_QUALITY_PIN);  
    float voltage = rawADC * (3.3 / 1023.0);  
  
    // 2. Determine Air Quality Status  
    String airStatus = getAirQualityStatus(rawADC);  
  
    // Numerical AQI Level Code: 1 = Good, 2 = Moderate, 3 = Poor  
    int statusCode = (rawADC < AQI_GOOD_LIMIT) ? 1 : ((rawADC <  
AQI_MODERATE_LIMIT) ? 2 : 3);
```

```
// 3. Local Serial Diagnostics Output
```

```
Serial.print("[AIR MONITOR] Raw ADC: ");
```

```
Serial.print(rawADC);
```

```
Serial.print(" | Voltage: ");
```

```
Serial.print(voltage);
```

```
Serial.print("V | Status: ");
```

```
Serial.println(airStatus);
```

```
// 4. Update ThingSpeak Fields
```

```
ThingSpeak.setField(1, rawADC); // Field 1: Raw Air Quality Index (0-1023)
```

```
ThingSpeak.setField(2, voltage); // Field 2: Sensor Output Voltage
```

```
ThingSpeak.setField(3, statusCode); // Field 3: Status Code (1=Good, 2=Moderate, 3=Poor)
```

```
// 5. Transmit Data to ThingSpeak Cloud
```

```
int httpCode = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
```

```
if (httpCode == 200) {
```

```
    Serial.println("[CLOUD] Telemetry pushed to ThingSpeak successfully.");
```

```
} else {
```

```
    Serial.print("[CLOUD ERROR] HTTP Post failed. Code: ");
```

```
    Serial.println(httpCode);
```

```
}
```

```
// ThingSpeak free tier requires >= 15 seconds delay
```

```
delay(20000);  
}
```

Coding output:

```
sketch_aug18c | Arduino IDE 2.3.10  
File Edit Sketch Tools Help  
Generic ESP8266 Mod...  
sketch_aug18c.ino  
17 // --- ThingSpeak channel details ---  
18 unsigned long myChannelNumber = 3458771;  
19 const char * myWriteAPIKey = "M9YMBVM273R9CG3D";  
20  
21 // --- Sensor configuration ---  
22 const int AIR_QUALITY_PIN = A0; // MQ-135 Analog Pin  
23  
24 // AQI Thresholds (Raw ADC values 0-1023)  
25 const int AQI_GOOD_LIMIT = 300; // Clean Air / Normal background  
26 const int AQI_MODERATE_LIMIT = 600; // Acceptable indoor air / slight pollutants  
27  
28 WiFiClient client;  
29  
30 // Helper function to convert AQI raw value into textual status  
31 String getAirQualityStatus(int rawVal) {  
32   if (rawVal < AQI_GOOD_LIMIT) {  
33     return "Good Air Quality";  
34   } else if (rawVal < AQI_MODERATE_LIMIT) {  
35     return "Acceptable indoor air / slight pollutants";  
36   } else {  
37     return "Unacceptable indoor air / high pollutants";  
38   }  
39 }  
40  
41 void setup() {  
42   Serial.begin(9600);  
43   pinMode(AIR_QUALITY_PIN, INPUT);  
44 }  
45  
46 void loop() {  
47   int rawVal = analogRead(AIR_QUALITY_PIN);  
48   float voltage = rawVal * 5.0 / 1023.0;  
49   String status = getAirQualityStatus(rawVal);  
50   Serial.print("[AIR MONITOR] Raw ADC: ");  
51   Serial.print(rawVal);  
52   Serial.print(" | Voltage: ");  
53   Serial.print(voltage);  
54   Serial.print(" | Status: ");  
55   Serial.println(status);  
56   if (client.connect("thingspeak.com", 80)) {  
57     String url = "/api/1/post?channel=" + myChannelNumber + "&field1=" + rawVal + "&key=" + myWriteAPIKey;  
58     client.print(url);  
59     Serial.println("Telemetry pushed to ThingSpeak successfully.");  
60   }  
61   delay(20000);  
62 }
```

Output Serial Monitor X
Message (Enter to send message to 'Generic ESP8266 Module' on 'COM10') New Line 9600 baud

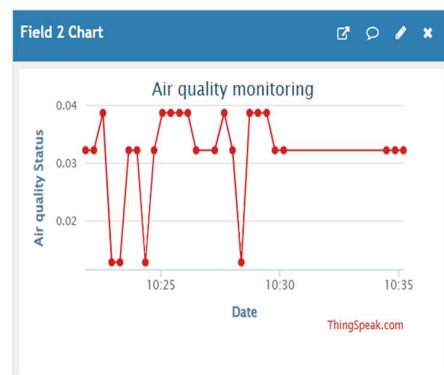
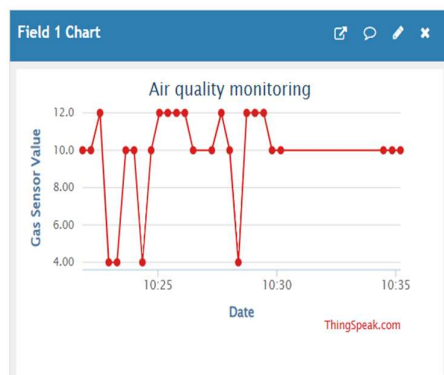
```
[WiFi] Connected successfully.  
[SENSOR] Warming up air quality heater element...  
[SENSOR] Sensor online. Starting sampling loop.  
  
[AIR MONITOR] Raw ADC: 10 | Voltage: 0.03V | Status: Good Air Quality  
[CLOUD] Telemetry pushed to ThingSpeak successfully.  
[AIR MONITOR] Raw ADC: 10 | Voltage: 0.03V | Status: Good Air Quality  
[CLOUD] Telemetry pushed to ThingSpeak successfully.  
[AIR MONITOR] Raw ADC: 10 | Voltage: 0.03V | Status: Good Air Quality  
[CLOUD] Telemetry pushed to ThingSpeak successfully.  
[AIR MONITOR] Raw ADC: 10 | Voltage: 0.03V | Status: Good Air Quality  
[CLOUD] Telemetry pushed to ThingSpeak successfully.  
[AIR MONITOR] Raw ADC: 10 | Voltage: 0.03V | Status: Good Air Quality  
[CLOUD] Telemetry pushed to ThingSpeak successfully.  
[AIR MONITOR] Raw ADC: 10 | Voltage: 0.03V | Status: Good Air Quality  
[CLOUD] Telemetry pushed to ThingSpeak successfully.
```

Channel Stats

Created: [about an hour ago](#)

Last entry: [less than a minute ago](#)

Entries: 26



System Working

- **Power ON:** The ESP8266 Node MCU and all connected components are powered ON.
- **Sensor Initialization:** The MQ2 Gas Sensor is initialized and allowed to warm up for stable operation.
- **Air Detection:** The MQ2 sensor continuously senses smoke and combustible gases present in the surrounding air.
- **Signal Generation:** The MQ2 sensor produces an analog output based on the detected gas level.
- **Data Reading:** The ESP8266 reads the MQ2 sensor output through the A0 analog input.
- **Data Processing:** The ESP8266 processes the sensor reading and compares it with the predefined threshold values.
- **Normal Condition:** If the gas level is within the normal range, the green LED turns ON, the red LED and buzzer remain OFF, and the display shows “NORMAL.”
- **Warning Condition:** If the gas level increases above the normal threshold, the red LED turns ON, the buzzer is activated, and the display shows “WARNING.”
- **Danger Condition:** If the gas level exceeds the danger threshold, the red LED and buzzer remain ON, and the display shows “DANGER.”
- **Display Update:** The current gas sensor value and pollution status are continuously updated on the LCD display.
- **Continuous Monitoring:** The ESP8266 repeatedly reads and processes the MQ2 sensor value to provide continuous monitoring.
- **System Stop:** The system stops monitoring when the power supply is switched OFF.

Bill of materials

S.No.	Component	Quantity
1	ESP8266 Node MCU	1
2	MQ2 Gas Sensor	1
3	Green LED	1
4	Red LED	1
5	Buzzer	1
6	Breadboard	1
7	Jumper Wires	Set 1
8	LED Resistors	2

Prototype implementation

The air pollution monitoring system prototype is implemented using an esp8266 nodemcu, mq2 gas sensor, two led indicators, and a buzzer. All the components are assembled on a breadboard and connected using jumper wires according to the designed circuit.

The mq2 gas sensor continuously detects smoke and combustible gases present in the surrounding air. Its analog output is connected to the a0 pin of the esp8266 nodemcu. The esp8266 reads and processes the sensor value and compares it with predefined threshold levels.

A green led is used to indicate a normal gas level, while a red led indicates an increased or dangerous gas level. The buzzer provides an audible warning when the gas level exceeds the defined threshold.

The control program is uploaded to the esp8266 using the arduino ide. After the mq2 sensor is initialized and warmed up, the esp8266 continuously reads the sensor output and controls the leds and buzzer based on the detected gas level.

The completed prototype provides a simple, low-cost, real-time gas and smoke detection system suitable for basic industrial, laboratory, indoor, and environmental safety demonstrations.

Hardware Setup:

The hardware setup of the Air Pollution Monitoring System consists of an ESP8266 Node MCU, MQ2 Gas Sensor, Green LED, Red LED, Buzzer, Breadboard, and Jumper Wires.

- Place the ESP8266 Node MCU on the breadboard as the main controller.
- Connect the MQ2 Gas Sensor to the breadboard.
- Connect the MQ2 AO pin to A0 of the ESP8266 for analog gas-level measurement.
- Connect the MQ2 GND to GND of the ESP8266.
- Connect the MQ2 power according to the specific sensor module's voltage requirement.
- Connect the Green LED to D5 (GPIO14) through a suitable current-limiting resistor.
- Connect the Red LED to D6 (GPIO12) through a suitable current-limiting resistor.
- Connect the Buzzer to D7 (GPIO13) and GND.
- Ensure that all components share a common ground.
- Upload the control program to the ESP8266 using the Arduino IDE.
- Power ON the circuit and allow the MQ2 sensor to warm up before taking readings.
- Test the system by observing the LED and buzzer response as the detected gas/smoke level changes.

Prototype Working:

The prototype works by continuously monitoring the surrounding air using the MQ2 Gas Sensor. The sensor detects changes in smoke and combustible gas levels and sends an analog signal to the ESP8266 Node MCU through the A0 pin.

The ESP8266 processes the sensor reading and compares it with predefined threshold values. When the gas level is within the normal range, the green LED turns ON, indicating safe conditions. When the gas level increases beyond the threshold, the red LED turns ON and the buzzer is activated, providing both visual and audible warnings.

The system continuously repeats the sensing and decision-making process, allowing it to provide real-time detection and warning of increased gas/smoke levels without using an LCD display.

Expected Prototype Output:

The prototype is expected to continuously monitor the surrounding air using the MQ2 Gas Sensor and provide visual and audible indications through the LEDs and buzzer.

Gas/Smoke Condition	MQ2 Sensor Reading	Green LED	Red LED	Buzzer	Expected Status
Normal	Low	ON	OFF	OFF	Safe
Warning	Medium	OFF	ON	ON	Warning
High Pollution	High	OFF	ON	ON	Danger

Expected Output:

- **Normal Condition:** Green LED glows and the buzzer remains OFF.
- **Warning Condition:** Red LED glows and the buzzer turns ON.
- **High Gas/Smoke Condition:** Red LED and buzzer remain ON to alert the user.
- The ESP8266 continuously monitors the MQ2 sensor value and automatically changes the LED and buzzer status according to the programmed threshold.

- The system provides a simple real-time visual and audible warning without using an LCD display.

Prototype Demonstration:

The prototype demonstration begins by powering ON the **ESP8266 Node MCU, MQ2 Gas Sensor, LEDs, and buzzer**. The MQ2 sensor is allowed to warm up and stabilize before testing. Under normal air conditions, the **green LED remains ON**, while the red LED and buzzer remain OFF.

During the demonstration, a controlled and safe amount of smoke or a suitable test source is introduced near the MQ2 sensor. As the sensor detects an increase in gas/smoke concentration, its analog output changes. The ESP8266 reads this change through the **A0 pin** and compares it with the programmed threshold.

When the sensor reading exceeds the threshold, the **green LED turns OFF, the red LED turns ON, and the buzzer produces an audible warning**. When the surrounding air returns to normal and the sensor reading falls below the threshold, the system returns to the normal indication.

The demonstration proves that the prototype can **detect changes in gas/smoke levels and provide immediate visual and audible alerts**. Testing should be performed in a well-ventilated area and without deliberately exposing anyone to hazardous gases.

Results & Performance Analysis

The developed Air Pollution Monitoring System using ESP8266 and MQ2 Gas Sensor successfully detects changes in smoke and combustible gas levels in the surrounding environment. The ESP8266 continuously reads the MQ2 sensor output and responds according to the predefined threshold values.

Expected Results:

Parameter	Result / Observation
Gas Detection	MQ2 successfully detects changes in smoke/gas levels
Controller Response	ESP8266 processes the sensor signal effectively
Normal Indication	Green LED turns ON
Warning Indication	Red LED turns ON
Audible Alert	Buzzer activates when the threshold is exceeded
Monitoring	Continuous real-time monitoring
Response	Quick response to changes in sensor reading
Reliability	Suitable for prototype-level monitoring
Cost	Low-cost implementation
Power Requirement	Suitable for a small embedded prototype
Calibration	Required for accurate quantitative gas measurement

Performance Analysis

The prototype provides effective real-time detection and alerting for changes in smoke and gas levels. The ESP8266 responds to variations in the MQ2 sensor output and controls the LEDs and buzzer accordingly.

The system is simple, economical, and suitable for demonstrating basic gas-monitoring functionality. However, the MQ2 is a general-purpose gas/smoke sensor, so its raw analog reading should not be considered a precise ppm measurement without proper calibration. Sensor warm-up time, temperature, humidity, supply voltage, and environmental conditions can also affect the readings.

Overall, the prototype demonstrates satisfactory performance for basic air-pollution and gas-level warning applications.